

# Ros2 Day2 hw2 AI 사용 보고서

휴머노이드 수습단원 이주원

## 목차

1. ai사용 부분
2. 대화내용
3. 깨달은 점

### 1. ai 사용 부분

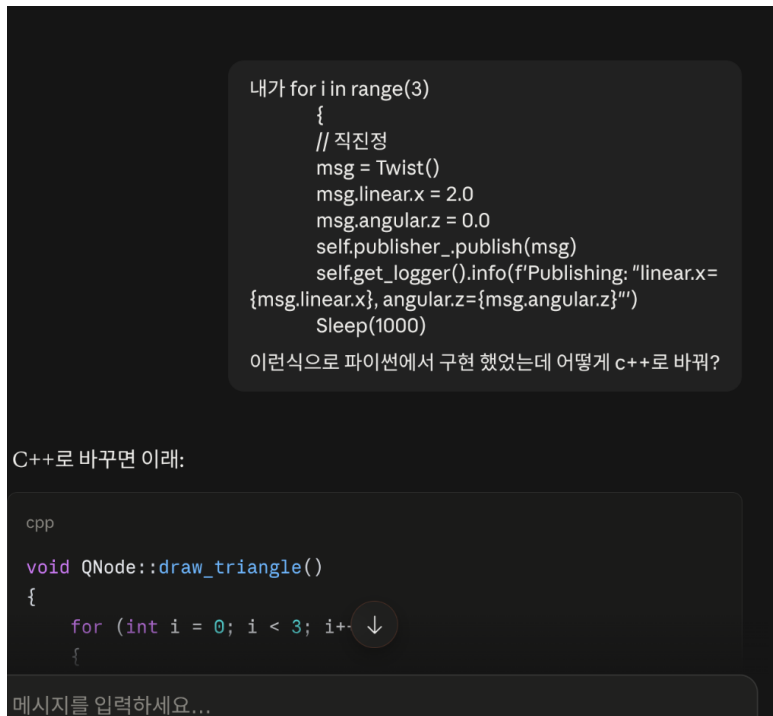
이 과제에서 ai가 사용된 부분은 첫 번째로 파이썬 코드를 C++로 옮기는 방법이다. Day1에서 파이썬으로 만들었던 도형 그리기를 그대로 옮기려 했는데, 메시지를 만들고 발행하는 부분과 대기하는 부분을 C++에서는 어떻게 써야 하는지 몰라서 물어보았다.

두 번째는 펜 설정이다. 파이썬에서 적용했던 펜의 색상과 굵기를 C++에서는 어떻게 적용해야 하는지 확인을 요청했다.

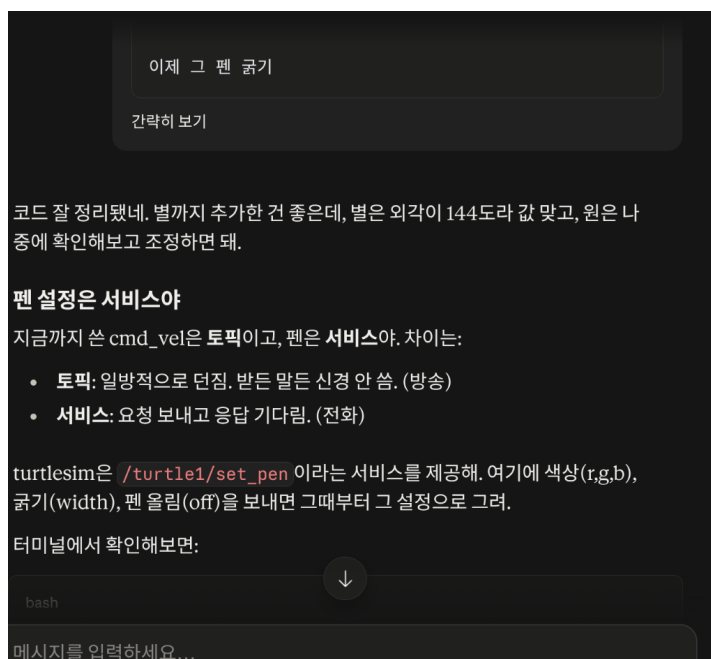
세 번째는 라벨 출력이다. 발행되는 속도 값을 화면의 라벨에 띄우려 했는데 어떻게 연결해야 하는지 몰라서 물어보았다.

네 번째는 그 라벨의 숫자가 바뀌지 않는 문제다. 연결은 해두었는데 도형을 그리는 동안 값이 갱신되지 않고 마지막 값만 남아 있어서, 혼자 원인을 찾아보다가 질문했다.

## 2. 대화내용



파이썬에서 쓰던 반복문과 발행 부분은 구조를 그대로 두고 문법을 어떻게 바꾸면 되는지 바꾸면 된다는 답을 받았다. 대기하는 부분은 파이썬과 달리 스레드 쪽 함수를 쓰면 된다는 것을 알았다.



펜 설정은 지금까지 쓰던 속도와 성격이 다르다는 답을 받았다. 속도 쪽은 토픽이라 받든 말든 상관없이 일방적으로 보내는 것이고, 펜 쪽은 서비스라 요청을 보내고 응답을 기다리는 방식이라고 한다. 터틀심이 제공하는 서비스에 색상과 굵기, 펜 올림 여부를 보내면 그때부터 그 설정으로 그린다는 설명을 들었다.

라벨

라벨 출력은 스레드 문제 때문에 한 단계 더 필요해.

## 왜 직접 못 하나

`qnode->publish_cmd_vel()` 은 QNode 스레드에서 실행돼. 근데 라벨은 GUI 객체라서 메인 스레드 소유야. 다른 스레드에서 `ui->label->setText()` 하면 프로그램이 뺏거나 화면이 깨져.

Qt는 이걸 위해 **시그널/슬롯**을 제공해. QNode가 "값 바뀌었어!"라고 신호(시그널)만 쓰면, Qt가 알아서 메인 스레드로 넘겨서 처리해줘.

## 1. QNode에 시그널 추가

`qnode.hpp` :

cpp

메시지를 입력하세요...

라벨 출력은 한 단계가 더 필요하다는 답을 받았다. 속도를 발행하는 부분은 통신 쪽 스레드에서 실행되는데 라벨은 화면 쪽 소유라, 다른 스레드에서 라벨을 직접 건드리면 프로그램이 뺏거나 화면이 깨질 수 있다고 한다. 그래서 통신 쪽에서는 값이 바뀌었다는 신호만 보내고 화면 쪽이 그 신호를 받아 라벨을 바꾸도록 해야 한다는 설명을 들었다.

저거 숫자가 안 바뀌어

잘 그려졌네. 도형 네 개 다 나오고 색상·굵기도 다 다르게 적용됐어.

라벨이 안 바뀌는 건 GUI가 얼어붙기 때문이야. 아까 말한 그 문제.

버튼을 누르면 `triangle()` 이 **메인 스레드**에서 실행돼. 그 안의 `msleep` 이 메인 스레드를 붙잡고 있어서, 시그널은 계속 날아가지만 Qt가 화면을 갱신할 틈이 없어. 도형 다 그리고 나서야 풀리는데 그땐 이미 마지막 `stop()` 의 `0 / 0` 이 찍혀 있는 거지. 지금 라벨에 `0 / 0` 인 게 딱 그 증거야.

터미널에 `Can only ... with QThread` 어쩌구 경고 뜬 것도 같은 맥락이야. 스레드 관련 경고.

## 해결

가장 간단한 건 라벨 갱신 후 이벤트를 강제로 처리하게 하는 거야.

`main_window.cpp` :

cpp

```
#include <QApplication>
```

메시지를 입력하세요...

라벨의 숫자가 바뀌지 않는 문제는 화면이 얼어붙어 있기 때문이라는 답을 받았다. 버튼을 누르면 도형을 그리는 함수가 화면 쪽 스레드에서 실행되는데, 그 안의 대기 부분이 화면 쪽을 붙잡고 있어서 신호는 계속 날아가지만 화면을 갱신할 틈이 없다는 것이다. 도형을 다 그리고 나서야 풀리는데 그때는 이미 마지막 정지 값이 찍혀 있어서, 라벨에 계속 0만 보였

던 것이 그 증거라고 한다. 터미널에 댄 스레드 관련 경고도 같은 맥락이라는 설명을 들었다. 해결로는 라벨을 갱신한 뒤 이벤트를 강제로 처리하게 하는 방법을 안내받았다.

### 3. 깨달은 점

GUI에서 화면을 담당하는 스레드와 그 외의 스레드가 나뉘어 있다는 것을 알게 되었다. 화면에 있는 것은 화면 쪽 스레드만 건드릴 수 있고, 다른 곳에서 직접 바꾸면 프로그램이 불안정해진다는 점이 인상 깊었다.

값이 제대로 전달되고 있어도 화면이 멈춰 있으면 보이지 않을 수 있다는 것을 알게 되었다. 연결이 잘못된 줄 알고 그쪽만 계속 봤는데 원인은 다른 곳이었다. 눈에 보이는 증상과 실제 원인이 다를 수 있다는 것을 배웠다.

대기하는 코드를 화면 쪽에서 실행하면 그동안 프로그램이 멈춘다는 것을 알게 되었다. 이번에는 이벤트를 강제로 처리하는 방법으로 넘어갔지만, 원래는 오래 걸리는 작업을 다른 스레드로 넘기는 것이 맞다는 점도 같이 알게 되었다.